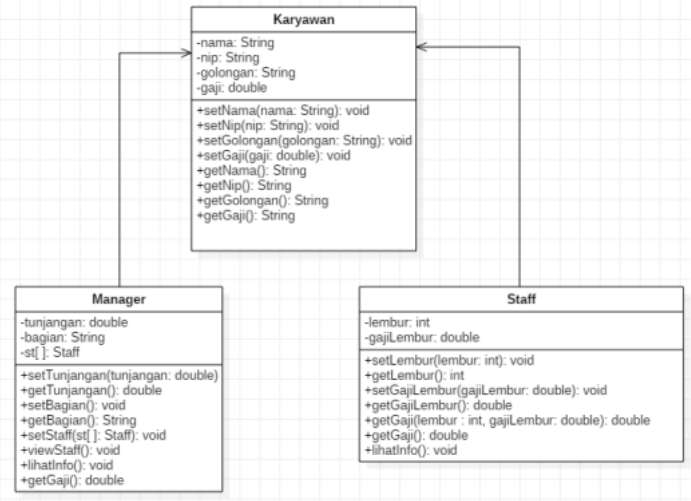
Jobsheet 7
Mata Kuliah Praktikum Pemrograman Berorientasi Objek
Overloading dan Overriding



Nama : **Rachmah Nur Chotimah**
NIM : **254107060052**
Kelas : **SIB 2F**

1. Percobaan 1



A. Karyawan

```java
public class Karyawan {
    private String nama;
    private String nip;
    private String golongan;
    private double gaji;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNip(String nip) {
        this.nip = nip;
    }

    public void setGolongan(String golongan) {
        this.golongan = golongan;
        switch (golongan.charAt(0)) {
            case '1': this.gaji = 5000000; break;
            case '2': this.gaji = 3000000; break;
            case '3': this.gaji = 2000000; break;
            case '4': this.gaji = 1000000; break;
            case '5': this.gaji = 750000; break;
        }
    }

    public void setGaji(double gaji) {
        this.gaji = gaji;
    }

    public String getNama() {
        return nama;
```

```
    }

    public String getNip() {
        return nip;
    }

    public String getGolongan() {
        return golongan;
    }

    public double getGaji() {
        return gaji;
    }
}
```

B. Staff

```
public class Staff extends Karyawan {
    private int lembur;
    private double gajiLembur;

    public void setLembur(int lembur) {
        this.lembur = lembur;
    }

    public int getLembur() {
        return lembur;
    }

    public void setGajiLembur(double gajiLembur) {
        this.gajiLembur = gajiLembur;
    }

    public double getGajiLembur() {
        return gajiLembur;
    }

    // Overloading method getGaji
    public double getGaji(int lembur, double gajiLembur) {
        return super.getGaji() + (lembur * gajiLembur);
    }

    // Overriding method getGaji
    @Override
    public double getGaji() {
        return super.getGaji() + (lembur * gajiLembur);
    }

    public void lihatInfo() {
```

```
        System.out.println("NIP        : " + this.getNip());
        System.out.println("Nama       : " + this.getNama());
        System.out.println("Golongan   : " + this.getGolongan());
        System.out.println("Jml Lembur : " + this.getLembur());
        System.out.printf("Gaji Lembur: %.0f\n",
this.getGajiLembur());
        System.out.printf("Gaji       : %.0f\n", this.getGaji());
    }
}
```

C. Manager

```
public class Manager extends Karyawan {
    private double tunjangan;
    private String bagian;
    private Staff st[];

    public void setTunjangan(double tunjangan) {
        this.tunjangan = tunjangan;
    }

    public double getTunjangan() {
        return tunjangan;
    }

    public void setBagian(String bagian) {
        this.bagian = bagian;
    }

    public String getBagian() {
        return bagian;
    }

    public void setStaff(Staff st[]) {
        this.st = st;
    }

    public void viewStaff() {
        System.out.println("--------------------");
        for (int i = 0; i < st.length; i++) {
            st[i].lihatInfo();
        }
        System.out.println("--------------------");
    }

    public void lihatInfo() {
        System.out.println("Manager  : " + this.getBagian());
        System.out.println("NIP      : " + this.getNip());
        System.out.println("Nama     : " + this.getNama());
```

```
        System.out.println("Golongan : " + this.getGolongan());
        System.out.printf("Tunjangan: %.0f\n", this.getTunjangan());
        System.out.printf("Gaji     : %.0f\n", this.getGaji());
        System.out.println("Bagian   : " + this.getBagian());
        this.viewStaff();
    }

    @Override
    public double getGaji() {
        return super.getGaji() + tunjangan;
    }
}
```

D. Utama

```
public class Utama {
    public static void main(String[] args) {
        System.out.println("Program Testing Class Manager & Staff");
        Manager man[] = new Manager[2];
        Staff staff1[] = new Staff[2];
        Staff staff2[] = new Staff[3];

        // Pembuatan Manager 1
        man[0] = new Manager();
        man[0].setNama("Tedjo");
        man[0].setNip("101");
        man[0].setGolongan("1");
        man[0].setTunjangan(5000000);
        man[0].setBagian("Administrasi");

        // Pembuatan Manager 2
        man[1] = new Manager();
        man[1].setNama("Atika");
        man[1].setNip("102");
        man[1].setGolongan("1");
        man[1].setTunjangan(2500000);
        man[1].setBagian("Pemasaran");

        // Staff untuk Manager 1
        staff1[0] = new Staff();
        staff1[0].setNama("Usman");
        staff1[0].setNip("0003");
        staff1[0].setGolongan("2");
        staff1[0].setLembur(10);
        staff1[0].setGajiLembur(10000);

        staff1[1] = new Staff();
        staff1[1].setNama("Anugrah");
```

```
        staff1[1].setNip("0005");
        staff1[1].setGolongan("2");
        staff1[1].setLembur(10);
        staff1[1].setGajiLembur(55000);

        man[0].setStaff(staff1);

        // Staff untuk Manager 2
        staff2[0] = new Staff();
        staff2[0].setNama("Hendra");
        staff2[0].setNip("0004");
        staff2[0].setGolongan("3");
        staff2[0].setLembur(15);
        staff2[0].setGajiLembur(5500);

        staff2[1] = new Staff();
        staff2[1].setNama("Arie");
        staff2[1].setNip("0006");
        staff2[1].setGolongan("4");
        staff2[1].setLembur(5);
        staff2[1].setGajiLembur(100000);

        staff2[2] = new Staff();
        staff2[2].setNama("Mentari");
        staff2[2].setNip("0007");
        staff2[2].setGolongan("3");
        staff2[2].setLembur(6);
        staff2[2].setGajiLembur(20000);

        man[1].setStaff(staff2);

        // Cetak informasi Manager + Staff
        man[0].lihatInfo();
        man[1].lihatInfo();
    }
}
```

E. Output

Program Testing Class Manager & Staff
Manager : Administrasi
NIP : 101
Nama : Tedjo
Golongan : 1
Tunjangan: 5000000
Gaji : 10000000
Bagian : Administrasi

```
        NIP       : 0003
        Nama      : Usman
        Golongan   : 2
        Jml Lembur : 10
        Gaji Lembur: 10000
        Gaji      : 3100000
        NIP       : 0005
        Nama      : Anugrah
        Golongan   : 2
        Jml Lembur : 10
        Gaji Lembur: 55000
        Gaji      : 3550000
        -------------------
        Manager  : Pemasaran
        NIP     : 102
        Nama     : Atika
        Golongan : 1
        Tunjangan: 2500000
        Gaji     : 7500000
        Bagian   : Pemasaran
        -------------------
        NIP       : 0004
        Nama      : Hendra
        Golongan   : 3
        Jml Lembur : 15
        Gaji Lembur: 5500
        Gaji      : 2082500
        NIP       : 0006
        Nama      : Arie
        Golongan   : 4
        Jml Lembur : 5
        Gaji Lembur: 100000
        Gaji      : 1500000
        NIP       : 0007
        Nama      : Mentari
        Golongan   : 3
        Jml Lembur : 6
        Gaji Lembur: 20000
        Gaji      : 2120000
--------------------
```

2. Latihan

```java
class Perkalianku {

    void perkalian(int a, int b) {

        System.out.println(a * b);

    }
```

```
    void perkalian(int a, int b, int c) {

        System.out.println(a * b * c);

    }

    public static void main(String args[]) {

        Perkalianku objek = new Perkalianku();

        objek.perkalian(25, 43);
        objek.perkalian(34, 23, 56);

    }
}
```

1. Dari source coding diatas terletak dimanakah overloading?
   Jawab: Overloading terletak pada method `perkalian()` dalam class Perkalianku. Yaitu antara method `void perkalian(int a, int b)` dan `void perkalian(int a, int b, int c)`.

2. Jika terdapat overloading ada berapa jumlah parameter yang berbeda?
   Jawab: Ada 2 variasi jumlah parameter, yaitu method pertama menggunakan 2 parameter `(int a, int b)`, sedangkan method kedua menggunakan 3 parameter `(int a, int b, int c)`.

```
class Perkalianku {

void perkalian(int a, int b){
    System.out.println(a * b);
}

void perkalian(double a, double b){
    System.out.println(a * b);
}

    public static void main(String args[]) {
        Perkalianku objek = new Perkalianku();

        objek.perkalian(25, 43);
        objek.perkalian(34.56, 23.7);

    }

}
```

3. Dari source coding diatas terletak dimanakah overloading?

Jawab: Overloading terletak pada method `perkalian()` dalam `class Perkalianku`. Yaitu antara method `void perkalian(int a, int b)` dan void `perkalian(double a, double b)`.

4. Jika terdapat overloading ada berapa tipe parameter yang berbeda?
   Jawab: Ada 2 tipe data parameter yang berbeda, yaitu bertipe int pada method pertama dan bertipe `double` pada method kedua.

```
class Ikan {
    public void swim() {
        System.out.println("Ikan bisa berenang");
    }
}

class Piranha extends Ikan {
    @Override
    public void swim() {
        System.out.println("Piranha bisa makan daging");
    }
}

public class Fish {
    public static void main(String[] args) {
        Ikan a = new Ikan();
        Ikan b = new Piranha();
        a.swim();
        b.swim();

    }
}
```

5. Dari source coding diatas terletak dimanakah overriding?
   Jawab: Overriding terletak pada method `swim()` di dalam subclass `Piranha` yang mendefinisikan ulang method `swim()` dari superclass `Ikan`.

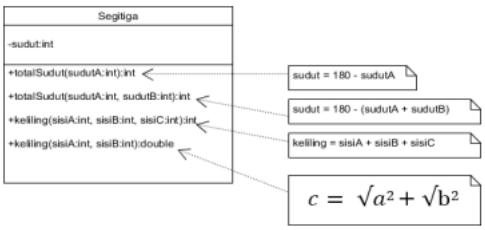
6. Jabarkanlah apabila sourcoding diatas jika terdapat overriding?

   Jawab: Pada kode tersebut, class `Piranha` meng-`extends` class `Ikan`. Class `Ikan` memiliki method `public void swim()` yang mencetak `"Ikan bisa berenang"`. Class `Piranha` meng-override method `swim()` tersebut sehingga mengubah perilakunya untuk mencetak `"Piranha bisa makan daging"`. Saat objek `Piranha` dipanggil melalui `Ikan b = new Piranha();`, method yang dieksekusi secara *dynamic binding / runtime polymorphism* adalah method milik `Piranha` (`b.swim()` mencetak `"Piranha bisa makan daging"`).

3. TUGAS

1. Overloading

Implementasikan konsep overloading pada class diagram dibawah ini :



Hasil Implementasi

```java
public class Segitiga {
    private int sudut;

    // Overloading Method 1
    public int totalSudut(int sudutA) {
        this.sudut = 180 - sudutA;
        return this.sudut;
    }

    // Overloading Method 2
    public int totalSudut(int sudutA, int sudutB) {
        this.sudut = 180 - (sudutA + sudutB);
        return this.sudut;
    }

    // Overloading Method 3
    public int keliling(int sisiA, int sisiB, int sisiC) {
        return sisiA + sisiB + sisiC;
    }

    // Overloading Method 4 (Rumus Phytagoras)
    public double keliling(int sisiA, int sisiB) {
        double c = Math.sqrt(Math.pow(sisiA, 2) + Math.pow(sisiB,
2));
        return sisiA + sisiB + c;
    }

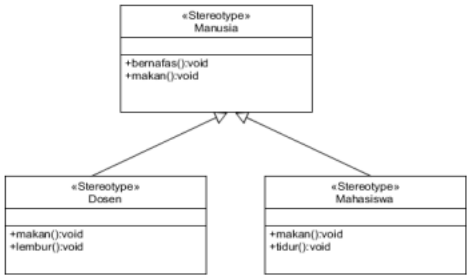
    public static void main(String[] args) {
        Segitiga sg = new Segitiga();
        System.out.println("Total Sudut (1 Sudut Diketahui) : " +
sg.totalSudut(60));
        System.out.println("Total Sudut (2 Sudut Diketahui) : " +
sg.totalSudut(60, 40));
        System.out.println("Keliling (3 Sisi)               : " +
sg.keliling(3, 4, 5));
        System.out.println("Keliling (2 Sisi Tegak Lurus)   : " +
sg.keliling(3, 4));
    }
}
```

```
Total Sudut (1 Sudut Diketahui) : 120
```

```
Total Sudut (2 Sudut Diketahui) : 80
Keliling (3 Sisi)               : 12
Keliling (2 Sisi Tegak Lurus)   : 12.0
```

2. Overriding

Implementasikan class diagram dibawah ini dengan menggunakan teknik dynamic method dispatch :




```java
// Parent class
class Manusia {
    public void bernafas() {
        System.out.println("Manusia bernafas menggunakan paru-
paru.");
    }

    public void makan() {
        System.out.println("Manusia makan makanan untuk bertahan
hidup.");
    }
}

// Child class Dosen
class Dosen extends Manusia {
    @Override
    public void makan() {
        System.out.println("Dosen makan siang di kantin kampus.");
    }

    public void lembur() {
        System.out.println("Dosen sedang lembur memeriksa tugas
mahasiswa.");
    }
}

// Child class Mahasiswa
class Mahasiswa extends Manusia {
    @Override
    public void makan() {
        System.out.println("Mahasiswa makan mie instan di kosan.");
    }
```

```
    public void tidur() {
        System.out.println("Mahasiswa tidur setelah merampungkan
tugas.");
    }
}

// Main class
public class MainManusia {
    public static void main(String[] args) {
        // Penerapan Dynamic Method Dispatch / Polymorphism
        Manusia m1 = new Manusia();
        Manusia m2 = new Dosen();     // Reference Manusia, Objek
Dosen
        Manusia m3 = new Mahasiswa(); // Reference Manusia, Objek
Mahasiswa

        System.out.println("=== MANUSIA ==");
        m1.bernafas();
        m1.makan();

        System.out.println("\n=== DOSEN (Dynamic Dispatch) ===");
        m2.bernafas();
        m2.makan(); // Memanggil method makan() milik Dosen

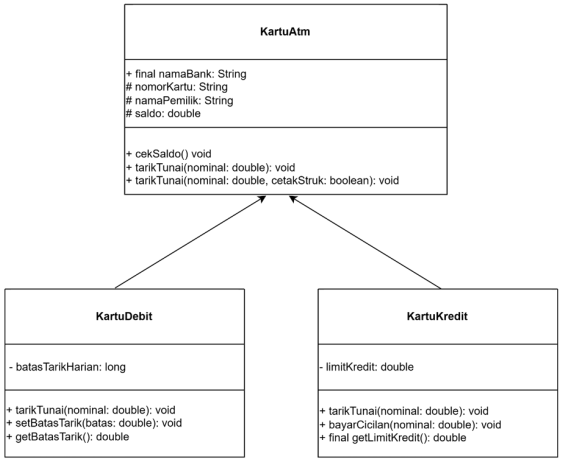
        System.out.println("\n=== MAHASISWA (Dynamic Dispatch) ===");
        m3.bernafas();
        m3.makan(); // Memanggil method makan() milik Mahasiswa
    }
}
```

```
=== MANUSIA ==
Manusia bernafas menggunakan paru-paru.
Manusia makan makanan untuk bertahan hidup.

=== DOSEN (Dynamic Dispatch) ===
Manusia bernafas menggunakan paru-paru.
Dosen makan siang di kantin kampus.

=== MAHASISWA (Dynamic Dispatch) ===
Manusia bernafas menggunakan paru-paru.
Mahasiswa makan mie instan di kosan.
```

3. **Class Diagram Hasil Tugas Teori PBO**



Hasil Implementasi

Parent

```
public class KartuAtm {
    // Atribut sesuai diagram
    public final String namaBank = "Bank Central"; // final attribute
    protected String nomorKartu;
    protected String namaPemilik;
    protected double saldo;

    // Constructor tanpa parameter
    public KartuAtm() {
    }

    // Constructor berparameter
    public KartuAtm(String nomorKartu, String namaPemilik, double saldo)
{
        this.nomorKartu = nomorKartu;
        this.namaPemilik = namaPemilik;
        this.saldo = saldo;
    }

    public void cekSaldo() {
        System.out.println("Bank          : " + namaBank);
        System.out.println("Nomor Kartu   : " + nomorKartu);
        System.out.println("Nama Pemilik  : " + namaPemilik);
        System.out.println("Saldo Saat Ini: Rp" + saldo);
    }

    // Overloading 1: Tarik tunai biasa
    public void tarikTunai(double nominal) {
        if (saldo >= nominal) {
            saldo -= nominal;
            System.out.println("Tarik tunai sebesar Rp" + nominal + "
berhasil.");
        } else {
```

```
            System.out.println("Saldo tidak mencukupi untuk tarik
tunai!");
        }
    }

    // Overloading 2: Tarik tunai dengan opsi cetak struk
    public void tarikTunai(double nominal, boolean cetakStruk) {
        this.tarikTunai(nominal);
        if (cetakStruk) {
            System.out.println("--- STRUK TRANSAKSI ---");
            System.out.println("Bank      : " + namaBank);
            System.out.println("Penarikan : Rp" + nominal);
            System.out.println("Sisa Saldo: Rp" + saldo);
            System.out.println("-----------------------");
        }
    }
}
```

Child 1

```
public class KartuDebit extends KartuAtm {
    private long batasTarikHarian;

    // Constructor tanpa parameter
    public KartuDebit() {
        super();
        this.batasTarikHarian = 5000000;
    }

    // Constructor berparameter
    public KartuDebit(String nomorKartu, String namaPemilik, double
saldo, long batasTarikHarian) {
        super(nomorKartu, namaPemilik, saldo);
        this.batasTarikHarian = batasTarikHarian;
    }

    // Overriding method tarikTunai
    @Override
    public void tarikTunai(double nominal) {
        if (nominal > batasTarikHarian) {
            System.out.println("Gagal! Nominal Rp" + nominal + " melebihi
batas tarik harian Rp" + batasTarikHarian);
        } else {
            super.tarikTunai(nominal);
        }
    }

    public void setBatasTarik(double batas) {
        this.batasTarikHarian = (long) batas;
    }
```

```
    public double getBatasTarik() {
        return (double) this.batasTarikHarian;
    }
}
```

Child 2

```
public class KartuKredit extends KartuAtm {
private double limitKredit;

    public KartuKredit() {
        super();
    }

    public KartuKredit(String nomorKartu, String namaPemilik, double
saldo, double limitKredit) {
        super(nomorKartu, namaPemilik, saldo);
        this.limitKredit = limitKredit;
    }

    // OVERRIDING: Tarik tunai kartu kredit biasanya memotong limit,
bukan saldo tabungan
    @Override
    public void tarikTunai(double nominal) {
        double biayaAdmin = 50000;
        double totalPotongan = nominal + biayaAdmin;

        if (limitKredit >= totalPotongan) {
            limitKredit -= totalPotongan;
            System.out.println("Tarik Tunai Kredit Rp" + nominal + "
(Admin Rp" + biayaAdmin + "). Sisa Limit: Rp" + limitKredit);
        } else {
            System.out.println("Gagal! Limit kredit tidak mencukupi.");
        }
    }

    public void bayarCicilan(double nominal) {
        limitKredit += nominal;
        System.out.println("Berhasil bayar cicilan Rp" + nominal + ".
Limit saat ini: Rp" + limitKredit);
    }

    // FINAL METHOD: Tidak bisa di-override oleh class turunannya di masa
depan
    public final double getLimitKredit() {
        return limitKredit;
    }
}
```

Main Class

```java
public class DemoKartu {
    public static void main(String[] args) {
        System.out.println("=== TEST OVERRIDING & OVERLOADING (KARTU
DEBIT) ===");
        KartuDebit debit = new KartuDebit("123-456", "Rachmah", 5000000,
2000000);
        debit.cekSaldo();

        // Memanggil Overriding (Mencoba tarik lebih dari batas harian)
        debit.tarikTunai(2500000);

        // Memanggil Overloading (Tarik tunai dengan cetak struk)
        debit.tarikTunai(1000000, true);

        System.out.println("\n=== TEST FINAL METHOD & OVERRIDING (KARTU
KREDIT) ===");
        KartuKredit kredit = new KartuKredit("987-654", "Rahma imooet",
0, 10000000);

        // Memanggil Overriding khusus Kartu Kredit
        kredit.tarikTunai(2000000);

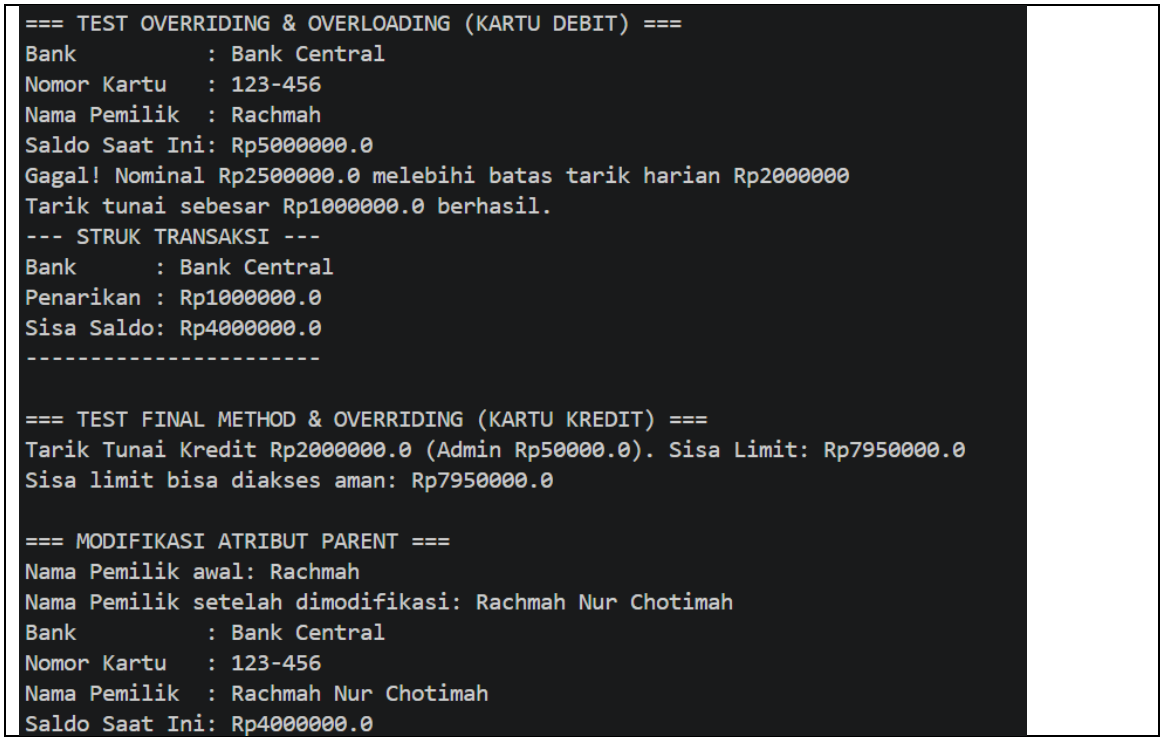
        // Memanggil Final Method
        System.out.println("Sisa limit bisa diakses aman: Rp" +
kredit.getLimitKredit());

        System.out.println("\n=== MODIFIKASI ATRIBUT PARENT ===");
        System.out.println("Nama Pemilik awal: " + debit.namaPemilik);

        // Modifikasi atribut yang diwariskan dari Parent
        debit.namaPemilik = "Rachmah Nur Chotimah";
        System.out.println("Nama Pemilik setelah dimodifikasi: " +
debit.namaPemilik);

        // Cek saldo lagi untuk membuktikan perubahannya
        debit.cekSaldo();
    }
}
```

Output



Link Github : https://github.com/lovvovala/PrakPBO/tree/main/Jobsheet_7